Vityarthi Project Report

Student marks predictor:

Pure Python multi-feature student marks predictor.

Name: Bhaskar Pathak

Reg. No: 25BCE11247

Branch: B-tech (CSE)

Subject: Fundamentals of AI/ML (CSA2001)

Professor: Suneet Joshi

Collage: Vit Bhopal

1. Problem Statement

As a student, I’ve noticed how hard it is to actually balance study time with things like sleep and attending classes. Everyone says "study more to get better marks," but they don't tell you how much sleep you're losing in the process or if missing that one morning lecture actually hurts your final score. I wanted to build a tool that doesn't just guess, but actually quantifies how these three things—study, sleep, and attendance—work together to decide a final grade.

---

2. Why This Problem Matters

This isn't just about numbers; it's about time management.

- Finding the Balance: If I stay up all night studying, my "Study Hours" go up but my "Sleep Hours" drop. I wanted a model that could show the point where being tired actually cancels out the extra studying.
- No "Black Boxes": Most AI tools today are like magic boxes where you don't see the math. By building this from scratch, I can see exactly how the "weights" shift when a student changes their habits.
- Realistic Planning: It helps in setting realistic goals. Instead of just "trying harder," a student can see that increasing attendance by 10% might actually be more effective than adding 2 more hours of study.

---

3. Approach to Solving the Problem

Since I wanted to understand the "soul" of the algorithm, I decided to code this without using any big libraries like Scikit-Learn or NumPy.

- The Math: I used Multiple Linear Regression. This lets the program look at three different inputs at the same time.
- The Learning Part: I implemented Stochastic Gradient Descent (SGD). Basically, the code starts with zero knowledge, makes a "guess" at the marks, sees how wrong it was (the error), and then slightly adjusts its internal "weights" to do better next time.
- The Loop: I ran this learning process for 10,000 cycles (epochs) so the model could really "memorize" the patterns in my training data.

---

4. Key Decisions Made

- Pure Python Only: I chose not to use libraries. It’s harder to write the math manually, but it proves I actually know how Gradient Descent works behind the scenes.
- Weight-based System: I decided to give each input (hours, sleep, attendance) its own "weight." This way, the AI can learn that maybe 1 hour of study is "heavier" (more important) than 1 hour of sleep for the final mark.
- Clamping the Output: I made sure the final prediction stays between 0 and 100. Mathematics can sometimes give weird results like 105% or -5%, so I added logic to keep it realistic for a marksheet.

---

5. Challenges Faced

- The "100% Error": At first, my code gave everyone 100% no matter what. I realized my "learning rate" was too high, causing the weights to explode. Lowering it to 0.0001 fixed everything and made the predictions much more accurate.

- Small Dataset: Since I didn't have a database of thousands of students, I had to carefully curate a small, realistic training set that covered extreme cases (like a student who studies a lot but never sleeps).
- Manual Math: Doing multi-variable math in a for loop without NumPy was tricky, especially keeping track of which weight belonged to which feature.

---

6. Code Organization and Documentation

I kept the code very modular so it's easy for anyone on GitHub to read:

- Train_model: This is the "brain" where all the heavy lifting and math happen.
- run_predictor: This is the "face" of the program that talks to the user and asks for their data.
- Interactive Feedback: Instead of just burying comments in the code, I used print statements that tell the user exactly what the AI is doing while it's training. It makes the "machine learning" part feel alive.
- Clean Naming: I used simple, human-readable variable names like rate, cycles, and error so even a non-coder could follow the logic.

Following is the output for the program I made:

```
--- Student Performance Analysis System ---
Daily study hours (0-15): 5
Average sleep hours (0-12): 7
Attendance percentage (0-100): 80

Analysis Result:
Predicted Final Mark: 75.06%
Verdict: Average Performer

Analyze another student? (y/n): n
System shutting down. Goodbye!
```